

# Module 6: GitHub & Collaboration

## 6.1 Working with Remotes

A **remote** is a version of your repository that lives on a server (like GitHub). It enables collaboration and backup.

### View your remotes:

```
git remote -v
```

### Output:

```
origin  https://github.com/username/repo.git (fetch)
origin  https://github.com/username/repo.git (push)
```

When you clone a repository, Git automatically names the remote **origin**.

### Add a remote:

```
git remote add origin https://github.com/username/repo.git
```

### Change a remote URL:

```
git remote set-url origin https://github.com/username/new-repo.git
```

### Remove a remote:

```
git remote remove origin
```

## 6.2 `git push` — Uploading to Remote

---

Push sends your **local commits** to the remote repository.

### First push (set upstream):

```
git push -u origin main
```

The `-u` flag sets `origin/main` as the **upstream** (default) for future pushes.

### Subsequent pushes:

```
git push
```

### Push a specific branch:

```
git push origin development
```

### Push all branches:

```
git push --all origin
```

### Force push (⚠️ dangerous):

```
git push --force
```

⚠️ **Force push overwrites the remote history.** Only use when you're absolutely sure (e.g., after a rebase on a personal branch).

---

## 6.3 `git pull` — Downloading from Remote

---

Pull **fetches** changes from the remote AND **merges** them into your current branch.

```
git pull origin main
```

Or simply (if upstream is set):

```
git pull
```

### What `git pull` actually does:

```
# git pull is equivalent to:  
git fetch origin    # Download changes  
git merge origin/main # Merge them into your branch
```

---

## 6.4 `git fetch` — Download Without Merging

Unlike `pull`, `fetch` downloads changes but **does NOT merge** them automatically.

```
git fetch origin
```

This is useful when you want to **review** the changes before merging:

```
# 1. Fetch the latest changes  
git fetch origin  
  
# 2. See what's different  
git diff main origin/main  
  
# 3. Merge when ready  
git merge origin/main
```

**git pull vs git fetch :**

| Command                | Downloads? | Merges automatically? | Risk level                |
|------------------------|------------|-----------------------|---------------------------|
| <code>git fetch</code> | ✓          | ✗                     | Safe — just downloads     |
| <code>git pull</code>  | ✓          | ✓                     | May cause merge conflicts |

**Tip:** Use `git fetch` when you want to be cautious. Use `git pull` when you want a quick update.

## 6.5 Forking a Repository

A **fork** is a personal copy of someone else's repository on your GitHub account.

### Why fork?

- You want to **contribute** to an open-source project
- You want to **experiment** without affecting the original
- You don't have write access to the original repo

### How to fork:

1. Go to the repository on GitHub
2. Click the **Fork** button (top right)
3. GitHub creates a copy under your account
4. Clone your fork:

```
git clone https://github.com/YOUR-USERNAME/forked-repo.git
```

### Keeping your fork updated:

```
# 1. Add the original repo as "upstream"
git remote add upstream https://github.com/ORIGINAL-OWNER/repo.git

# 2. Fetch from upstream
git fetch upstream

# 3. Merge upstream changes into your branch
git merge upstream/main
```

## 6.6 Pull Requests (PRs)

A **pull request** is a GitHub feature that lets you **propose changes** to a repository. It's the standard way to contribute code in collaborative projects.

### The Pull Request Workflow:

```
graph TD
    classDef step fill:#e1f5fe,stroke:#0288d1,stroke-width:2px,color:#000;
    classDef action fill:#e8f5e9,stroke:#388e3c,stroke-width:2px,color:#000;

    A["1. Create Local Branch<br/>'git checkout -b feature'"]:::step --> B["2. Commit Changes<br/>'git commit -m feature'"]:::step
    B --> C["3. Push to Remote Fork/Repo<br/>'git push origin feature'"]:::step
    C --> D["4. Open Pull Request on GitHub<br/>(Compare & PR)"]:::action
    D --> E["5. Code Review & Testing<br/>(Approvals/Changes)"]:::action
    E --> F["6. Merge Pull Request to 'main'"]:::step

    style F fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px,color:#000
```

## Detailed Step-by-Step Pull Request Flow:

| Step                     | Scope / Action                                                                | Status of Repository                                                                  | Git / GitHub Internal Mechanics                                                                                                                             |
|--------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Create Branch</b>  | Local: <code>git checkout -b feature/improvement</code>                       | Active branch shifted to <code>feature/improvement</code> .                           | Git creates <code>.git/refs/heads/feature/improvement</code> pointing to the current commit, and updates <code>HEAD</code> .                                |
| <b>2. Commit Changes</b> | Local: <code>git add .</code><br><code>git commit -m "Add new feature"</code> | Working directory clean; branch tip moved forward.                                    | Blobs/trees written; new local commit created.                                                                                                              |
| <b>3. Push to GitHub</b> | Local: <code>git push origin feature/improvement</code>                       | Local and GitHub feature branch are now synchronized.                                 | Git uploads commits and objects to GitHub. GitHub creates the remote branch <code>feature/improvement</code> on their servers.                              |
| <b>4. Open PR</b>        | GitHub Web: Click "Create Pull Request"                                       | PR is open and marked "Open" status.                                                  | GitHub scans the diff between the feature branch and the base branch (e.g. <code>main</code> ), checks for merge conflicts, and displays a discussion page. |
| <b>5. Code Review</b>    | GitHub Web: Team reviews and comments                                         | Review approvals/change requests are recorded.                                        | Reviewers comment directly on specific code lines. Automated tests (CI/CD workflows) can run via GitHub Actions.                                            |
| <b>6. Merge PR</b>       | GitHub Web: Click "Merge Pull Request"                                        | The feature branch code is integrated into <code>main</code> . Remote branch deleted. | GitHub performs a merge (three-way or squash) on the server, updates the <code>main</code> branch pointer, and closes the pull request.                     |

## PR Best Practices:

| Practice                   | Why                                          |
|----------------------------|----------------------------------------------|
| Write clear titles         | Reviewers understand the purpose at a glance |
| Add a detailed description | Context helps reviewers understand the "why" |
| Keep PRs small             | Easier to review, fewer conflicts            |
| Link related issues        | Creates traceability                         |
| Request specific reviewers | Gets the right eyes on your code             |

## 6.7 .gitignore — Excluding Files from Tracking

A `.gitignore` file tells Git which files and folders to **ignore** (not track).

### Create a `.gitignore` file:

Create a file named `.gitignore` in the **root** of your repository:

```
touch .gitignore
```

## Common .gitignore patterns:

```
# Dependencies
node_modules/
vendor/

# Environment variables (sensitive data!)
.env
.env.local

# Build output
dist/
build/
*.min.js

# OS-generated files
.DS_Store
Thumbs.db

# IDE/Editor files
.vscode/
.idea/
*.swp

# Logs
*.log
npm-debug.log*

# Compiled files
*.class
*.o
*.pyc
__pycache__/
```

## Pattern syntax:

| Pattern                     | Meaning                                                             |
|-----------------------------|---------------------------------------------------------------------|
| <code>filename</code>       | Ignore a specific file                                              |
| <code>folder/</code>        | Ignore an entire folder                                             |
| <code>*.ext</code>          | Ignore all files with that extension                                |
| <code>!important.log</code> | Do NOT ignore this file (exception)                                 |
| <code>**/logs</code>        | Ignore <code>logs</code> folders anywhere in the project            |
| <code>doc/*.txt</code>      | Ignore <code>.txt</code> files only in the <code>doc/</code> folder |

## Already tracking a file you want to ignore?

If a file was committed before adding it to `.gitignore` :

```
# Remove from tracking but keep on disk
git rm --cached filename.txt

# Then commit the change
git add .gitignore
git commit -m "Added .gitignore and stopped tracking filename.txt"
```

**Tip:** GitHub offers ready-made `.gitignore` templates for many languages and frameworks at [github.com/github/gitignore](https://github.com/github/gitignore).

## 6.8 GitHub Features Overview

| Feature       | Description                               |
|---------------|-------------------------------------------|
| Issues        | Track bugs, features, and tasks           |
| Pull Requests | Propose and review code changes           |
| Actions       | Automate workflows (CI/CD)                |
| Projects      | Kanban-style project boards               |
| Wiki          | Documentation for your project            |
| Discussions   | Community Q&A forum                       |
| Releases      | Package and distribute versions           |
| Pages         | Host static websites directly from a repo |

## 6.9 SSH vs HTTPS Authentication

### HTTPS:

```
git clone https://github.com/username/repo.git
```

- Requires username/password or **Personal Access Token (PAT)**
- Easier to set up
- Token needs to be refreshed periodically

### SSH:

```
git clone git@github.com:username/repo.git
```

- Uses SSH key pair (public/private)
- No need to enter credentials each time
- More secure for frequent use

## Setting up SSH:

```
# 1. Generate an SSH key
ssh-keygen -t ed25519 -C "your-email@example.com"

# 2. Start the SSH agent
eval "$(ssh-agent -s)"

# 3. Add your key
ssh-add ~/.ssh/id_ed25519

# 4. Copy the public key
cat ~/.ssh/id_ed25519.pub
# (Copy the output)

# 5. Add to GitHub:
#   GitHub → Settings → SSH Keys → New SSH Key → Paste
```

## Key Takeaways

1. **git push** uploads local commits to the remote; **git pull** downloads and merges
2. **git fetch** downloads without merging — safer for reviewing first
3. **Forking** creates a personal copy of someone else's repo
4. **Pull Requests** are the standard way to propose and review changes
5. **.gitignore** prevents unnecessary files from being tracked
6. Use **SSH** for passwordless authentication, **HTTPS** for quick setup